# SwarmSentinel: Claude UI Handoff and MCP Proxy Options

Claude Code session, 2026-10-04 · Repository: /Users/deepamk/source/aivillage/swarmsentinel (branch main)

**USER**

```
! pwd
/Users/deepamk/source/aivillage/swarmsentinel
```

**CLAUDE**

You're in the SwarmSentinel repo root, `/Users/deepamk/source/aivillage/swarmsentinel`, on `main`. The only uncommitted change is the modified v3 deck, `docs/source-materials/SwarmSentinel-v3---Live-Containment.pptx`. What do you want to work on?

**USER**

the project we have worked here - how do I transition this knowledge to Claude UI ?

**CLAUDE**

That depends on what you want to do in the web UI. There are two quite different options.

**If you want to keep building the code, use Claude Code on the web (claude.ai/code).** It runs against your GitHub repo, so it reads `CLAUDE.md` automatically the way this session does. Point it at `github.com/prasad-m-k/swarmsentinel` and the engine rules, data-handling rules and git rules come along. Three things won't come along:

- **Your auto-memory.** It holds two rules: no Claude co-author trailer, and never overwrite a deliverable you've edited. Add both to `CLAUDE.md` if you want them on the web.
- **Anything gitignored.** That means `data/ai-village/`, `download-hugginface/` and `graphify-out/`. This is correct: the raw dataset and SQLite store should never leave your machine. Replays against real AI Village data stay a local-only job.
- **The macOS pnpm workaround.** You won't need it, because the cloud environment is Linux, which matches the committed lockfile.

**If you want to discuss, write or present the project (papers, CFPs, decks, judging Q&A), use a claude.ai Project.** Set it up in four steps:

1. **Project instructions.** Paste the parts of `CLAUDE.md` that matter outside a terminal: what SwarmSentinel is, the two data paths, the hard data-handling rules, the framing rules (counterfactual language, ASP as a proposal), the dataset citation, and your writing rules. Leave out the commands, the layout table and the macOS caveats.

2. **Project knowledge.** Use the GitHub integration to attach selected repo files rather than the whole repo: `README.md`, `CLAUDE.md`, `docs/THREAT_MODEL.md`, the two policy JSONs, the dataset access and handling docs, and the core Python files if you want code-level answers. Upload the ASP preprint PDF and the current deck by hand. `graphify-out/GRAPH_REPORT.md` is also worth uploading as a compact map of the code, though it's gitignored and will go stale as the code changes.
3. **Never upload** raw dataset files, the SQLite store or AI Village trace exports. That would break the same rule that keeps them out of git. Aggregates and the structure-only snapshots are fine.
4. **Memory.** Tell claude.ai's memory the deliverable-overwrite rule once, or put it in the project instructions.

Neither route carries over the conversation history from the sessions where we built this. What survives is what's written down in `CLAUDE.md`, the docs, the memory files and the git log. Details that only came up in chat are lost unless we write them down now.

I can put the claude.ai route together for you: draft the project instructions, already trimmed and following your writing rules, plus a list of which files to attach. I'd also fold the two memory rules into `CLAUDE.md` so the code route picks them up. Which route are you after, or do you want both?

**USER**

How easy/difficult to build MCP Proxy? on top of existing solution?

**CLAUDE**

An MVP is easy. A proxy you could trust in production is a medium-sized job. The hard part is not the plumbing. It's that MCP doesn't carry most of the information the gateway's checks depend on.

### Why the base is easy

The gateway already does what a proxy needs. `ASPGateway.evaluate_and_log` takes one normalized tool-call `Event` and returns an allow, throttle or drop decision with a rule and a reason. `sdk/guard.py` wraps that for both in-process use (`Guard.local()`) and remote use (`Guard.remote()`). It already tracks lineage, completion receipts and an idempotency journal.

The naming fits too: `mock.asp.json` already uses targets like `mock:mcp:wiki.write`, so `mcp:<server>:<tool>` slots in without changes.

An MCP proxy is mostly a translation layer:

1. It presents itself as an MCP server to the client.
2. It connects to the real servers downstream.
3. On `tools/call`, it builds an `Event` and asks the Guard.
4. On a drop, it returns a tool result with `isError: true` and the rule and reason in the text. The downstream server is never called.
5. On `tools/list`, it can hide tools the policy would deny anyway.

With the official Python `mcp` SDK, a stdio-only version is a few hundred lines, about a day or two of work.

### Where it gets harder

**Identity and lineage.** MCP has no notion of which agent is calling or who spawned it, and lineage is the first check in Algorithm 1. The simplest mapping is one proxy session per client connection, with each connection registered as an agent under a root. That works for one agent. Swarm-level detection only works when several agents' clients go through the same proxy and you can tell them apart, for example with a header or per-agent config on HTTP transport. Spawn and delegation have no MCP equivalent at all, so scope narrowing would have to come from config.

**Missing event fields.** `detail`, `network`, `reads`, `write` and `resource` feed the argument constraints, the network allowlist, content-trust taint and the write cap.

- `detail` can be the serialized arguments.
- `network` has to be pulled from arguments per tool, such as a `url` field.
- `write` could come from MCP's `readOnlyHint` and `destructiveHint` annotations, but servers set those themselves, so they're only hints.
- Under the engine rule that behaviour lives in policy, the clean answer is a new `mcp` section in the policy JSON. It would map each server or tool to its trust level, its write flag and the argument field that holds the network destination, all validated by `ASPPolicy`. This is the bulk of the real design work.

**Content trust.** Taint currently spreads through `message` events between agents. Through a proxy you only see tool calls, so you can taint a session when it calls a tool marked untrusted, like a web fetch. You can't see cross-agent propagation unless agents talk to each other through MCP tools.

**Calls you can't see.** If an agent also has native tools or a shell outside the proxy, those calls bypass it. That's the `out_of_band` channel the gateway already models as "observed, cannot block". The proxy gives you a process boundary, which is stronger than today's in-process boundary, but still only covers what is routed through it. The threat model should say this plainly.

**Rest of the protocol.** Resources, prompts and sampling would initially be passed through with no checks. They need to be listed as uncovered in `THREAT_MODEL.md` so the status column stays honest.

**Transport and auth.** Streamable HTTP with several clients, auth and session lifecycle is where most of the remaining days go.

### Rough effort

| Version | Scope | Effort |
|---|---|---|
| MVP | stdio, one agent, local Guard, `mcp:*` targets, deny rendered as a tool error, a few tests | 1 to 2 days |
| Usable | adds the `mcp` policy section, `tools/list` filtering, untrusted-tool taint, `source: "mcp"` added to `Event` and `openapi.yaml`, Live tab support | about a week |
| Multi-agent over HTTP | per-agent identity, auth, shared session so swarm detection works, threat-model updates | 1 to 2 weeks |

The existing latency benchmark (`examples/bench_latency.py`) suggests the gateway won't be the bottleneck. The synthetic regression tests wouldn't be affected, since every change is additive.

If you want, I can build the MVP on a feature branch: a `proxy/` package next to `sdk/`, wrapping one real MCP server such as the filesystem reference server, with a test that shows a denied call never reaching it.

**USER**

> So, plugging the gw inside mcp itself as in-process task is helpful?

**CLAUDE**

It helps in one specific case, and it costs you the main thing that makes this a swarm gateway rather than a per-tool filter.

The specific case is an MCP server you write yourself. If its tool handlers ask `Guard.local()` before running, that's the `@agent.tool(...)` decorator pattern the SDK already supports, just placed on the server side. There is a real gain here. The tool author knows whether the tool writes, which argument holds a URL, and whether its output comes from untrusted sources. So `write`, `network` and `reads` get declared accurately instead of guessed from hints. That fixes the biggest weakness of the proxy approach. There's also no extra network hop, and nothing extra to deploy.

The cost is that each server keeps its own separate gateway state. Most of what SwarmSentinel adds over a plain allowlist depends on seeing every action in one place:

- **Content-trust taint breaks.** In the typical injection path, an agent reads an untrusted page through a fetch server, then writes through a filesystem server. With a gateway inside each server, the fetch server knows the context is tainted but never sees the write. The filesystem server sees the write but never saw the read. Neither one can block it.
- **Swarm-wide counters split.** The write cap, repeated-intent detection and step budgets are each counted per server, so an agent spreading its activity across servers stays under every limit.
- **Revocation stops working.** When Sentinel revokes an agent, that decision has to reach every place the agent can act. Separate in-process gateways each keep their own revoked set.
- **Identity is still missing.** A server receives a tool call without knowing which agent made it, so the lineage problem doesn't go away.
- **Third-party servers are out of reach.** You can't add the hook to servers you don't control.

- **The server polices itself.** If the server is buggy or compromised, it can simply skip the check. ASP's model puts the check outside the component it constrains.

So in-process is the right call, but the process should be the proxy, not each server. One proxy process runs `Guard.local()` and sits in front of all the MCP servers. That gives you one place where taint, counters and revocations live, still without a network hop. For servers you own, you can get the accurate tool descriptions without splitting state: have them declare their semantics in the `mcp` policy section, or report to the same engine session through `Guard.remote()`. Third-party servers go behind the proxy with policy-declared semantics. Either way, every decision lands in one session, and that session is what the dashboard and Sentinel need.

**USER**

Give me weightage pros/cons of in-process/proxy/both, with advantages and disadvantages

**CLAUDE**

Below I score three options. **In-process** means each MCP server runs its own `Guard.local()` inside its tool handlers. **Proxy** means one proxy process runs `Guard.local()` in front of every server, with tool semantics declared in a new `mcp` section of the policy JSON. **Both** means everything still goes through the proxy, which does all the enforcing, while the servers you own also hand the proxy a manifest of their real tool semantics, and can optionally run a second check of their own. Scores run from 1 to 5, and the weighted total is out of 100.

| Criterion | Weight | In-process | Proxy | Both |
|---|---|---|---|---|
| Cross-server content-trust taint | 20 | 1 | 4 | 5 |
| Swarm-wide counters and revocation | 15 | 1 | 5 | 5 |
| Accuracy of tool semantics (write, network, reads) | 15 | 5 | 3 | 5 |
| Coverage of third-party servers | 10 | 1 | 5 | 5 |
| Check sits outside what it constrains | 10 | 2 | 4 | 4 |
| Agent identity and lineage | 10 | 1 | 3 | 3 |
| Latency and operational simplicity | 10 | 5 | 4 | 3 |
| Build effort | 10 | 4 | 3 | 2 |
| **Weighted total** | **100** | **48** | **78** | **84** |

The weights reflect what makes SwarmSentinel different from a per-tool allowlist. Taint plus swarm-wide counters and revocation take 35 of the 100 points. That is why in-process scores poorly even though it is the cheapest option.

**In-process (48).** Its strengths are that the tool author knows exactly what each tool does, there's no extra network hop, nothing new to deploy, and the SDK's `@tool` decorator already does most of the work. Its weakness is that state is split across servers. Taint from a read in one server is invisible to a

write in another. Write caps, repeated-intent detection and step budgets are counted per server, so an agent can stay under every limit by spreading its calls. A revocation reaches only the one server that made it. It can't cover third-party servers, and a buggy or compromised server can skip its own check. It's fine for guarding one server you own and nothing more.

**Proxy (78).** It holds all state in one process, so taint, counters, revocation and the dashboard session all work as they do today. It covers any MCP server without changing that server, and the check runs outside the thing it constrains. The cost is that tool semantics come from policy declarations or server-supplied hints, so a tool you've labelled wrong gets judged wrong. Identity is per connection, which is coarse. There's one extra local hop. Calls that skip the proxy, like native tools or a shell, can only be observed. Resources, prompts and sampling would pass through unchecked at first.

**Both (84).** It keeps the proxy's single state and coverage, and adds accurate semantics for the servers you own, so their writes and untrusted reads are labelled correctly instead of guessed. The cost is two layers to keep in sync: if a manifest goes stale, the proxy enforces against the wrong labels. A compromised server you own could also misdescribe its own tools, for example by claiming a write is read-only. That's why it doesn't score higher on the outside-the-component criterion. It also takes the most to build and test.

On these weights, Both comes out ahead of Proxy by 6 points. If you weight delivery speed more heavily, say for a hackathon deadline, Proxy wins instead. Both is really Proxy plus a manifest format, so it doesn't have to be chosen up front. I'd build the proxy first and add manifests only for the servers you own, once there's a real tool whose policy labels turn out to be wrong. The in-process approach should stay a convenience inside those servers, never the only enforcement layer.

**USER**

save the current conversation in this window as pdf. I will review it later.